

MANUAL26. УРОК 12. ЖИЗНЕННЫЙ ЦИКЛ ДЕФЕКТА: КАК ПРОВЕСТИ БАГ ОТ ОБНАРУЖЕНИЯ ДО ЗАКРЫТИЯ.

Почему после оценки важности дефекта работа только начинается?

На прошлом уроке вы научились определять severity и priority. Но даже идеальная оценка не спасает, если задача теряется между ролями. Именно поэтому следующий шаг — понять, как дефект проходит путь от «нашли» до «проверили и закрыли».

Что такое жизненный цикл дефекта в реальной команде?

Это рабочий маршрут задачи, который помогает всем участникам быстро ответить на три вопроса: где задача сейчас, кто делает следующий шаг и когда ждать обновление. Если один из этих ответов размытый, дефект начинает «висеть» и терять приоритет в реальной работе.

Почему жизненный цикл важен даже в маленькой студии?

Потому что в маленькой команде один человек часто совмещает несколько ролей, и без прозрачного процесса задачи теряются еще быстрее. На словах кажется, что «и так договоримся», а на деле к релизу всплывают старые баги со статусом «в работе» без движения неделями.

Какие риски появляются без статусов и переходов?

Обычно команда сталкивается с повторяющимся набором проблем:

1. один и тот же дефект обсуждают по кругу;
 2. непонятно, кто следующий владелец задачи;
 3. нет ясности, в какой сборке проверять фикс;
 4. задачи закрываются без верификации;
 5. спорные дефекты живут месяцами в подвешенном состоянии.
-

Как выглядит базовая цепочка дефекта?

Чаще всего она похожа на
`New/Open` → `Accepted/Triage`
→ `In Progress` → `Fixed` → `In QA` → `Verified` → `Closed`.
Названия в Jira/YouTrack могут немного отличаться, но смысл один: каждая стадия должна фиксировать не факт активности, а результат шага.

Что означает каждый ключевой статус?

1. New/Open — баг зафиксирован и описан.
2. Accepted/Triage — команда признала дефект и определила приоритет.
3. In Progress — разработка взяла задачу в работу.

4. Fixed — изменения внесены, нужен retest.
5. In QA — дефект на проверке.
6. Verified — исправление подтверждено в целевой среде.
7. Closed — задача закрыта после верификации.

Если проблема осталась, задача переходит в `Reopened` и цикл повторяется.

Почему статус `Need Info` часто становится «кладбищем задач»?

Потому что его ставят без владельца и срока. Сам по себе этот статус полезен, когда реально не хватает данных, но он должен сопровождаться конкретикой: что именно нужно, кто приносит и когда возвращаемся к решению.

Что важно указывать в задаче, чтобы она не застряла в самом начале?

Минимум для живого дефекта:

1. шаги воспроизведения;
2. фактический и ожидаемый результат;
3. окружение (платформа, версия, сборка);
4. частота воспроизведения;
5. артефакты (видео, скриншот, логи при необходимости).

Если этого нет, команда тратит время на уточнения, а не на решение.

Почему Fixed не означает «вопрос закрыт»?

Потому что исправление в коде не равно исправлению в пользовательском сценарии. До retest дефект считается риском. После retest может выясниться частичная поломка, побочный эффект или проблема в другом окружении.

Как должен выглядеть хороший комментарий разработчика на этапе Fixed?

В нем полезно указать:

1. номер сборки, где есть фикс;
2. кратко, какая зона изменялась;
3. что QA особенно важно перепроверить;
4. есть ли известные ограничения.

Так QA быстрее и точнее проводит retest.

Когда нужно использовать Reopened?

Когда дефект воспроизводится после фикса или исправлен только частично. При переоткрытии нельзя писать просто «не работает». Нужно добавить, что именно осталось, в какой среде, как часто и чем отличается от предыдущей версии проблемы.

Как это выглядит на примере Genshin Impact?

В событии игроки иногда не получают экран награды. QA заводит дефект с видео. На triage задача получает приоритет. Разработчик делает фикс и указывает сборку. QA

повторно проверяет в исходной среде и в смежной. На Android проблема исчезла, но на iOS осталась в 2 из 10 повторов — значит Reopened с уточненными условиями, а не «закрыто с оговорками».

Почему качество переходов важнее количества закрытых задач?

Потому что метрика «много Closed» ничего не стоит, если часть задач закрыта формально. Для релиза важна не скорость закрытия статусов, а подтвержденный результат в пользовательских сценариях.

Какие сигналы говорят, что процесс в команде здоровый?

Обычно видно следующие признаки:

1. у каждого статуса есть понятный владелец;
2. переходы сопровождаются коротким объяснением;
3. задачи не живут неделями без движения;

4. retest проводится в внятном окне времени;
 5. споры по дефектам решаются фактами, а не авторитетом.
-

Как QA может ускорить цикл без давления на команду?

Рабочие действия простые:

1. писать воспроизводимо и без «воды»;
2. заранее уточнять данные для проверки фикса;
3. ежедневно смотреть задачи без прогресса;
4. на triage выносить проблемы с фактами;
5. фиксировать договоренности письменно в тикете.

Это уменьшает количество «пинг-понга» между ролями.

Что дает внутренний SLA в процессе дефектов?

SLA делает цикл предсказуемым. Например: новая критичная задача получает реакцию в течение дня; `Fixed` по P0/P1 уходит на retest в приоритетное

окно; задачи в `Need Info` пересматриваются не позже чем через 24 часа. Это не наказание, а понятный рабочий ритм.

Какие ошибки чаще всего делают начинающие QA?

Типовой список:

1. переводят статус без комментария;
 2. закрывают задачу по устному «должно работать»;
 3. переоткрывают без новых данных;
 4. забывают назначить следующего владельца;
 5. не фиксируют в задаче принятые решения;
 6. путают «исправили код» и «подтвердили сценарий».
-

Как проверить качество перехода перед сменой статуса?

Перед каждым переходом задайте короткий чек:

1. понятно ли, что именно произошло;
2. кто делает следующий шаг;

3. есть ли доказательства перехода;
4. поймет ли человек вне контекста, что и почему изменили.

Если ответа нет хотя бы на два пункта, статус менять рано.

Какой главный вывод урока?

Жизненный цикл дефекта — это не бюрократия, а механизм командной ответственности. Он защищает релиз от «невидимых» потерь: недопроверенных фиксов, забытых задач и несогласованных решений.

Какой мостик к следующему уроку?

Когда вы умеете проводить баг по статусам, следующий шаг — проверять последствия изменений. В уроке 13 разберем регрессию: как убедиться, что фикс в одной зоне не сломал соседние сценарии.

Как отличить «живой» процесс

дефектов от формального?

Живой процесс узнается по предсказуемости: у задач есть движение, у каждого перехода есть объяснение, у команды нет постоянных вопросов «а что с этим багом?». Формальный процесс выглядит красиво в трекере, но не дает результата: статусы меняются, а риски остаются.

Какие роли кроме QA влияют на жизненный цикл?

На практике цикл дефекта — командный:

1. QA создает и ведет воспроизводимый контекст;
2. разработка уточняет технические условия и вносит фикс;
3. продукт/лид помогает приоритизировать в рамках релиза;
4. иногда DevOps/аналитика дают данные по окружению.

Если хотя бы одна роль «выпадает», задача тормозится.

Что делать с дефектами, которые «не воспроизводятся»?

Это частая ситуация в игровых и сетевых продуктах. Важно не закрывать задачу автоматически. Лучше перевести в режим управляемого уточнения:

1. расширить окружение и условия;
2. собрать дополнительные артефакты;
3. проверить частоту на нескольких средах;
4. договориться о сроке повторного анализа.

Так вы снижаете риск потерять плавающий, но опасный дефект.

Когда допустим статус `Won't Fix` и как не злоупотреблять им?

`Won't Fix` возможен, когда цена исправления несоразмерна риску и есть согласованная бизнес-причина. Но это решение должно быть прозрачным: почему не исправляем, какой риск принимаем и кто утвердил

решение. Иначе статус превращается в «склад неудобных задач».

Почему важно хранить историю переходов внутри задачи?

Потому что через неделю или месяц команда может забыть логику принятых решений. История переходов помогает быстро восстановить контекст: что было подтверждено, что уточнили, почему приоритет менялся и кто принимал финальное решение.

Как не допускать «пинг-понга» между QA и разработкой?

Пинг-понг обычно возникает из-за недосказанности. Снизить его помогают:

1. конкретные критерии retest;
2. точное окружение фикса;
3. короткая фиксация «что проверено / что осталось»;
4. единый формат комментариев при `Reopened`.

Чем меньше недоговоренности,
тем меньше бесполезных циклов.

Как анализировать «застрявшие» задачи системно?

Раз в день полезно смотреть
задачи без движения и делить их
на типы:

1. не хватает данных;
2. не назначен owner;
3. нет актуальной сборки для проверки;
4. конфликт приоритетов.

После такого разбора triage
становится предметным: не
«почему стоит», а «что
конкретно мешает».

Какой дополнительный практический результат дает урок?

Вы должны уметь не только
провести задачу по статусам, но
и:

1. вовремя выявлять узкие места процесса;
2. формулировать причины задержек в терминах

- действий;
- 3. защищать необходимость верификации перед закрытием;
- 4. поддерживать прозрачность цикла для всей команды.

Это повышает качество релиза сильнее, чем просто «быстрее закрывать дефекты».

Как связать жизненный цикл дефекта с качеством коммуникации в команде?

Чем прозрачнее переходы, тем меньше скрытых ожиданий между ролями. Если разработка понимает критерий `Fixed`, а QA понимает критерий `Verified`, количество конфликтов резко снижается. Поэтому процесс дефектов — это не только трекер, но и общий язык взаимодействия.

Почему полезно периодически делать аудит статусов?

Раз в несколько спринтов полезно смотреть, какие статусы

реально используются, где задачи стоят дольше всего и какие переходы чаще всего возвращаются назад. Такой аудит помогает убрать «мусорные» этапы и усилить слабые места процесса.

Что полезно закрепить перед практикой по статусам?

Сфокусируйтесь на трех вещах: четкие критерии перехода, явный owner следующего шага и короткая письменная фиксация. Именно это чаще всего отличает рабочий цикл дефекта от формального.

Как проверить, что процесс не разваливается в релизный пик?

Если даже в перегруженную неделю у задач сохраняется понятный owner и не растёт число «висящих» статусов без комментария, значит жизненный цикл дефекта в команде действительно устойчив.

Глоссарий к занятию

1. **Жизненный цикл дефекта**
— последовательность шагов, через которые проходит дефект от обнаружения до закрытия.
2. **Переход статуса** — смена состояния задачи в процессе работы команды.
3. **Owner** — ответственный за ближайшее действие по задаче.
4. **Retest** — повторная проверка дефекта после исправления.
5. **Reopened** — повторное открытие задачи, если проблема сохранилась.
6. **SLA обработки** — внутренняя договоренность по времени реакции на ключевые этапы работы с дефектом.